SC2002 / SC2302 - Group Project

Restaurant Rush

How everything works

A complete guide to the game, the code and the design: the rules, every class, all three stages, the UML diagrams, the test runs and how they map to the assignment brief.

STAGE 1

Basic operation

Regular customers, Waiter, Chef, Cashier, 18 turns

STAGE 2

Customer behaviour

VIPCustomer and CriticCustomer

STAGE 3

New role + menu policy

Host, ComboMeal, Happy Hour

Java command-line game | stage-1/, stage-2/, stage-3/ | due Sunday 15 November 2026, 11:59 PM SGT

Contents

Contents . . . 2

1. The game in one page . . . 4

A customer's journey . . . 4

2. How a turn works . . . 5

Why the order of steps gives the right timing . . . 5

Worked example from the brief (one Regular customer ordering Pasta) . . . 5

3. How the code is organised . . . 7

Every class and what it is responsible for . . . 7

Three design decisions that make everything else simple . . . 7

4. Following one turn through the code . . . 9

How an invalid choice is handled . . . 9

A staff method: validate, then act . . . 10

Payment: oldest first, one per turn . . . 10

5. Stage 1 - Basic restaurant operation . . . 11

What Stage 1 contains . . . 11

Files . . . 11

The Customer class: ready to be extended, without any later-stage features . . . 11

Stage 1 class diagram . . . 12

Stage 1 test runs . . . 13

6. Stage 2 - Customer behaviour . . . 14

New requirement . . . 14

The key idea: polymorphism, not type checks . . . 14

What changed from Stage 1 to Stage 2 . . . 15

Stage 2 class diagram . . . 16

Stage 2 sequence diagram - serving a VIP from arrival to payment . . . 17

Stage 2 sequence diagram - part 2 of 3: turn 5 . . . 18

Stage 2 sequence diagram - part 3 of 3: turns 6 to 8 and the result check . . . 19

How the sequence diagram meets the brief . . . 21

Stage 2 test runs . . . 21

7. Stage 3 - New role and menu policy . . . 22

7.1 Host . . . 22

7.2 ComboMeal . . . 22

7.3 Happy Hour . . . 22

What changed from Stage 2 to Stage 3 . . . 24

Stage 3 class diagram (final design) . . . 25
Stage 3 test runs . . . 26
8. The three stages side by side . . . 27
Effect on the design (not just line counts) . . . 27
9. OO concepts and design principles . . . 28
Object-oriented concepts . . . 28
SOLID - only where it is genuinely in the code . . . 28
Trade-offs and an alternative we considered . . . 28
How the player's choices change the result . . . 28
10. Every rule in the brief, and where the code enforces it . . . 29
11. Running the game and the submission checklist . . . 30
Build and run any stage . . . 30
How to play . . . 30
Project layout . . . 30
Submission checklist (from the briefing) . . . 30

1. The game in one page

Restaurant Rush is a turn-based restaurant management game played in the terminal. **You are the manager.** Customers arrive, wait for a table, order, eat and pay. Your staff can only do a little each turn, so every turn you decide *who gets served first*. After 18 turns the game checks whether you hit three targets.

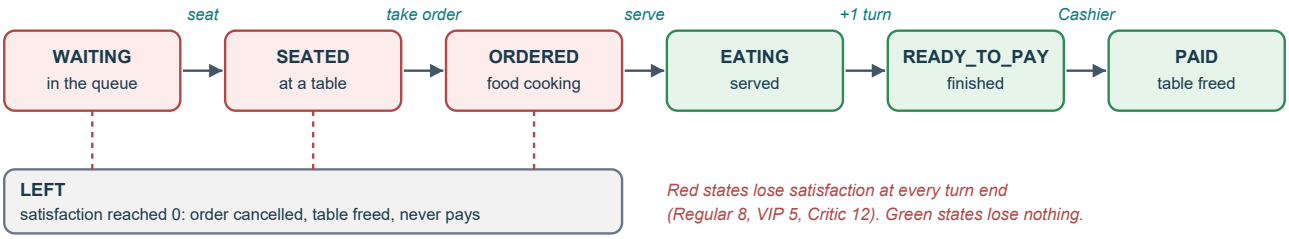
Setting	Base value (always available, so a TA can reproduce runs)
Turns	18, always played to the end (no early finish)
Tables	2 (a table stays occupied until its customer pays or leaves)
Staff	1 Waiter (one task per turn), 1 Chef (one preparation unit per turn), 1 Cashier (one payment per turn, automatic)
Arrivals	One customer at the start of turns 1, 4, 7, 10, 13 and 16 (C1 ... C6)
Win (all three)	**at least 4 paid customers** AND **revenue at least $45.00** AND **average satisfaction of paid customers at least 60**

Menu item	Price	Prep units	Stored as
Salad	$9.00	1	`new MenuItem("Salad", 900, 1)`
Burger	$15.00	1	`new MenuItem("Burger", 1500, 1)`
Pasta	$18.00	2	`new MenuItem("Pasta", 1800, 2)`

Money is stored as whole **cents** (an `int`) everywhere, so there are no rounding errors. `Money.format(1350)` turns it into "$13.50" for display.

A customer's journey

Every customer moves through the same states (the `CustomerState` enum). The manager's staff choices are what move them forward. While a customer is still waiting (red) they lose satisfaction at the end of every turn. If it reaches 0 they leave without paying.



Customers choose their own food

The manager never picks the dish. Rule used (deterministic, documented): **customer number n orders menu item number (n - 1) mod menuSize**. With the three-item menu: C1 Salad, C2 Burger, C3 Pasta, C4 Salad, C5 Burger, C6 Pasta. (The Stage 2 Critic overrides this rule; see Section 6.)

2. How a turn works

Every turn runs the same six steps, in this exact order. The order matters: it is what makes the timing rules in the brief come out right without any special cases.

#	Step	What happens	Where in the code
0	Display	Show turn, revenue, average satisfaction, queue, tables, each customer's satisfaction and order, staff tasks available.	`ConsoleUI.showTurnStart()`
1	Arrivals	The scheduled customer joins the waiting queue with satisfaction 100.	`ArrivalSchedule.customerArrivingOn()` `Restaurant.addCustomer()`
2a	Waiter	Manager picks ONE task: seat a queued customer, take a seated customer's order, serve a READY order, or wait.	`GameController.waiterStep()` `Waiter.seat / takeOrder / serve`
2b	Chef	Manager picks ONE order to advance by one unit, or wait. Runs after the Waiter, so the Chef can cook an order taken this turn.	`GameController.chefStep()` `Chef.prepare()`
3	Payment	Cashier takes at most one payment: the oldest READY_TO_PAY customer. Adds bill to revenue, marks PAID, frees the table.	`Cashier.collectPayment()`
4	Eating	Anyone served on an EARLIER turn finishes eating and becomes READY_TO_PAY.	`Customer.finishEating()`
5	Waiting	Everyone not yet served (WAITING, SEATED, ORDERED) loses satisfaction. At 0 they leave: order cancelled, table freed.	`Customer.loseSatisfaction()` `Customer.leaveUnhappy()`
6	Result	After turn 18 only: VICTORY if all three targets are met, otherwise DEFEAT.	`GameController.isVictory()`

Why the order of steps gives the right timing

Rule in the brief	How the step order guarantees it
A dish finished this turn can be served next turn at the earliest.	The Waiter (step 2a) acts **before** the Chef (2b). When the Chef finishes a dish, the Waiter has already used their task.
The Chef may work on an order the Waiter took this turn.	Same reason: the order already exists when the Chef step runs.
A customer served this turn eats for one full turn.	Eating (step 4) only finishes customers whose `servedTurn < turn`.
A customer who became READY_TO_PAY this turn pays next turn at the earliest.	Payment (step 3) runs **before** eating (step 4), so a customer who just finished eating is not yet READY_TO_PAY when the Cashier looks.
Served, eating or ready-to-pay customers lose nothing.	`isWaitingForService()` is only true for WAITING, SEATED and ORDERED.

Worked example from the brief (one Regular customer ordering Pasta)

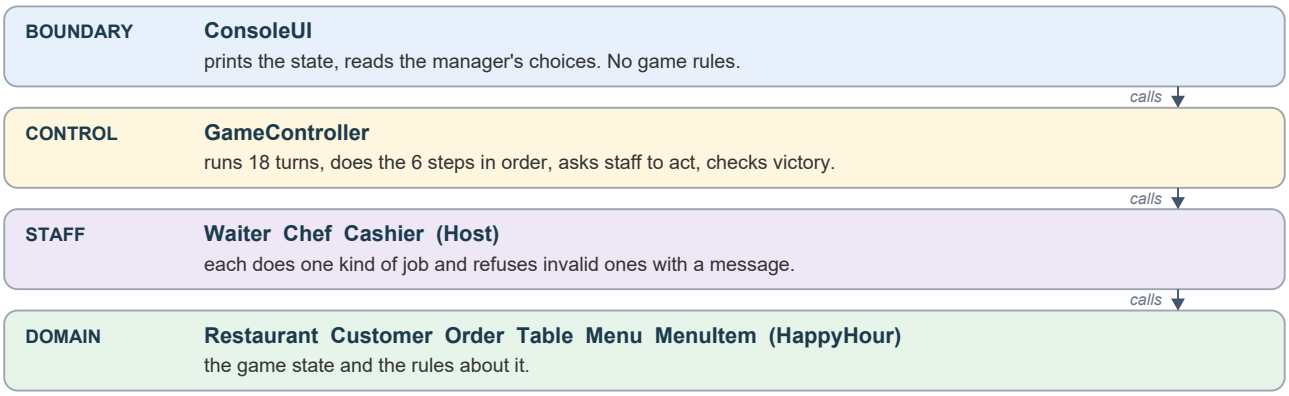
Our code reproduces the brief's Section 3.3 example exactly. Satisfaction drops by 8 at the end of every turn until the food arrives, then stays fixed and is recorded when the customer pays.

Turn	What happens	State after	Satisfaction at turn end
1	Arrives (100). Waiter seats the customer.	SEATED	92
2	Waiter takes the Pasta order. Chef does unit 1 of 2.	ORDERED, order 1/2	84
3	Chef does unit 2 of 2: order READY. Waiter cannot serve it this turn.	ORDERED, order READY	76
4	Waiter serves.	EATING, order SERVED	76 (no further loss)
5	Customer finishes eating.	READY_TO_PAY	76

Turn	What happens	State after	Satisfaction at turn end
6	Cashier collects $18.00: PAID, table freed.	PAID	76 recorded

3. How the code is organised

The design follows the **boundary / control / entity** split from the course. The rule that keeps it simple: **the UI never contains game rules**, and the controller never contains customer-specific rules.



Every class and what it is responsible for

Class	Kind	Responsibility	Since
`Main`	setup	Builds the menu and restaurant (base setting) and starts the game.	1
`GameController`	control	Runs the 18 turns and the six steps in order. Asks the UI for choices, hands them to staff, checks victory.	1
`ConsoleUI`	boundary	The only class that prints or reads. Re-asks if the input is not a number. No game rules.	1
`Restaurant`	entity	The game state: menu, tables, customers (in arrival order), revenue. Seats customers, computes statistics.	1
`ArrivalSchedule`	entity	Decides who arrives on which turn (and, from Stage 2, of which type).	1
`Customer`	entity	Satisfaction, state, table, order. Owns the rules that differ between customer types.	1
`Table`	entity	Holds at most one customer.	1
`Order`	entity	Item, locked price, preparation progress, status.	1
`Menu`, `MenuItem`	entity	What can be ordered: name, price in cents, preparation units.	1
`Waiter`, `Chef`, `Cashier`	staff	Each performs its own task and refuses invalid ones with a clear message.	1
`Money`	utility	Formats cents as dollars; (Stage 2+) percentage with rounding to the nearest cent.	1
`CustomerState`, `OrderStatus`	enums	Named states instead of magic strings.	1
`VIPCustomer`, `CriticCustomer`	entity	Customer subclasses with their own satisfaction loss, bill or food choice.	2
`Host`	staff	Seats one queued customer per turn, in addition to the Waiter.	3
`ComboMeal`	entity	A MenuItem made of two MenuItems (90% of the price, sum of the units).	3
`HappyHour`	policy	Once-per-game, two-turn promotion: 20% off new orders, +15 satisfaction for waiting customers.	3

Three design decisions that make everything else simple

Decision	Why
The waiting queue is derived, not stored. It is "customers in state WAITING, in arrival order" (`Restaurant.getWaitingQueue()`).	No second list to keep in sync. "Oldest first" for the Cashier is just a loop over customers in arrival order.
Invalid choices throw an exception with a message. Staff methods throw `IllegalArgumentException`; the controller catches it, shows it, and asks again.	A mistake never uses up the task (a brief requirement), and every check lives next to the rule it protects.
Customer-type rules are methods on Customer from Stage 1: `getSatisfactionLoss()`, `calculateBill()`, `chooseItem()`.	Stage 2 then only needs subclasses that override them. The controller never asks "what type is this customer?".

4. Following one turn through the code

This is the heart of the game: `GameController.playTurn()` from Stage 1. Each comment matches a step in Section 2. Read it top to bottom and you have read the rules.

GameController.playTurn() (stage-1)

```
private void playTurn(int turn) {
    // Step 1: arrivals
    Customer arrival = schedule.customerArrivingOn(turn);
    if (arrival != null) {
        restaurant.addCustomer(arrival);
    }
    ui.showTurnStart(turn, restaurant);
    if (arrival != null) {
        ui.showMessage(">> " + arrival + " arrives and joins the queue.");
    }

    // Steps 2 + 3: manager chooses; Waiter acts before Chef
    waiterStep(turn);
    chefStep();

    // Step 4: payment, then eating
    String payment = cashier.collectPayment(restaurant);
    if (payment != null) {
        ui.showMessage(">> Cashier: " + payment);
    }
    for (Customer c : restaurant.getCustomersInState(CustomerState.EATING)) {
        if (c.getServedTurn() < turn) { // served on an earlier turn
            c.finishEating();
            ui.showMessage(">> " + c + " finished eating and is READY_TO_PAY.");
        }
    }

    // Step 5: waiting customers lose satisfaction and may leave
    for (Customer c : restaurant.getCustomers()) {
        if (c.isWaitingForService()) {
            c.loseSatisfaction();
            if (c.getSatisfaction() == 0) {
                c.leaveUnhappy();
                ui.showMessage(">> " + c + " got fed up and LEFT without paying.");
            }
        }
    }
    ui.showTurnEnd(turn, restaurant);
}
```

How an invalid choice is handled

The Waiter step keeps asking until the choice works. If a staff method throws, the message is shown and the loop goes round again, so the task is not used up. The Chef step (and in Stage 3 the Host step) works the same way.

GameController.waiterStep() (stage-1)

```
private void waiterStep(int turn) {
    while (true) {
        int task = ui.askWaiterTask();
        if (task == 0) {
            ui.showMessage(">> Waiter waits.");
            return;
        }
        if (task < 0 || task > 3) {
            ui.showError("Unknown waiter task " + task + ".");
            continue;
        }
        int id = ui.askCustomerId();
        try {
            String result;
            if (task == 1) {
                result = waiter.seat(restaurant, id);
            } else if (task == 2) {
                result = waiter.takeOrder(restaurant, id);
            } else {
                result = waiter.serve(restaurant, id, turn);
            }
            ui.showMessage(">> Waiter: " + result);
            return;
        } catch (IllegalArgumentException e) {
            ui.showError(e.getMessage());
        }
    }
}
```

A staff method: validate, then act

The Waiter checks the rule, then lets the customer object do the work. Note that the Waiter asks the customer what it wants (`chooseItem`) and what it pays (`calculateBill`): this is the hook that Stage 2 builds on.

Waiter.takeOrder() (stage-1)

```
/** The customer picks their own dish; the price is locked now. */
public String takeOrder(Restaurant restaurant, int customerId) {
    Customer customer = restaurant.findCustomer(customerId);
    if (customer == null || customer.getState() != CustomerState.SEATED) {
        throw new IllegalArgumentException("Customer " + customerId + " is not seated and waiting to order.");
    }
    MenuItem item = customer.chooseItem(restaurant.getMenu());
    int price = customer.calculateBill(item.getPriceCents()); // each customer type prices its own bill
    customer.placeOrder(new Order(item, price));
    return customer + " orders " + item.getName() + " for " + Money.format(price) + ".";
}
```

Payment: oldest first, one per turn

Cashier.java (stage-1)

```
/** The Cashier automatically takes ONE payment per turn, oldest customer first. */
public class Cashier {

    /** Returns a message, or null if nobody was ready to pay. */
    public String collectPayment(Restaurant restaurant) {
        for (Customer customer : restaurant.getCustomers()) { // arrival order = oldest first
            if (customer.getState() == CustomerState.READY_TO_PAY) {
                int bill = customer.getOrder().getPriceCents();
                restaurant.addRevenue(bill);
                customer.pay();
                return customer + " pays " + Money.format(bill) + " (satisfaction " + customer.getSatisfaction() + "). Table free
            }
        }
        return null;
    }
}
```

5. Stage 1 - Basic restaurant operation

> **Development assumption**
> Only the Stage 1 requirements are known. Build and test a complete baseline game.

What Stage 1 contains

- Regular customers and the three menu items (Salad, Burger, Pasta).
- Waiter, Chef, Cashier, Table, Order and the waiting queue.
- The full turn sequence: arrivals, choices, cooking, serving, eating, payment, satisfaction loss, abandonment, victory and defeat.
- The player sets seating / ordering / serving priority through the Waiter and kitchen priority through the Chef.
- **16 classes, 656 lines** in `stage-1/src`. No VIP, Critic, Host, combo or Happy Hour code at all.

Files

Group	Files in stage-1/src
Control and boundary	`Main`, `GameController`, `ConsoleUI`
Game state	`Restaurant`, `ArrivalSchedule`, `Table`
Customers and orders	`Customer`, `Order`, `CustomerState`, `OrderStatus`
Menu	`Menu`, `MenuItem`, `Money`
Staff	`Waiter`, `Chef`, `Cashier`

The Customer class: ready to be extended, without any later-stage features

Stage 1's `Customer` already exposes the three behaviours that differ between customer types as overridable methods, with the Regular values. The brief allows "general support for future extensions"; there are no subclasses yet.

Customer.java (stage-1, excerpt)

```
// ----- behaviour that customer types can change -----

public String getType() { return "Regular"; }

/** Satisfaction lost for every turn spent waiting. */
public int getSatisfactionLoss() { return 8; }

/** The price this customer pays for a dish. */
public int calculateBill(int basePriceCents) { return basePriceCents; }

/** Rule: customer #n picks menu item number (n-1) mod menuSize. */
public MenuItem chooseItem(Menu menu) {
    return menu.getItem((id - 1) % menu.size());
}
```

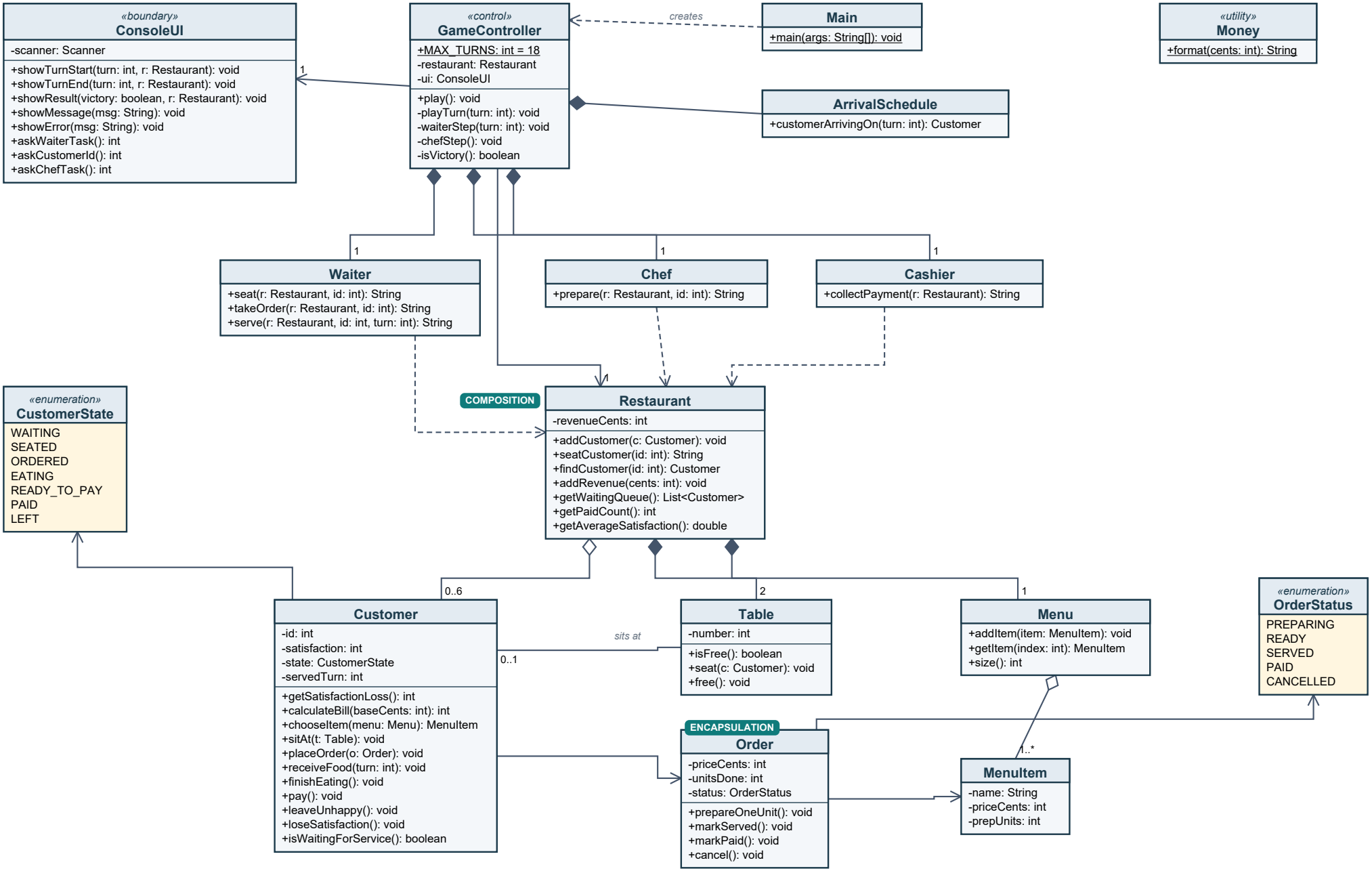
Customer.loseSatisfaction()

```
public void loseSatisfaction() {
    satisfaction = Math.max(0, satisfaction - getSatisfactionLoss());
}
```

Encapsulation in practice: satisfaction is `private` and only changes through `loseSatisfaction()`, which never lets it go below 0. An `Order`'s status only changes through `prepareOneUnit()`, `markServed()`, `markPaid()` and `cancel()`, and its price is `final`.

Stage 1 class diagram

Matches `stage-1/src`. Filled diamond = composition (owner creates and owns the part), hollow diamond = aggregation, open arrow = association, dashed arrow = dependency (uses). Underlined members are static.



Stage 1 test runs

Both runs are scripted input files, so anyone can reproduce them exactly:

```
cd stage-1
javac -d out src/*.java
java -cp out Main < tests/victory.txt
java -cp out Main < tests/defeat.txt
```

Victory run (`stage-1/tests/victory.txt`): one customer at a time, served as fast as the rules allow.

Customer	Dish	Arrive	Seated	Ordered	READY	Served	Ready to pay	Paid	Bill	Satisf.
C1	Salad	1	1	2	2	3	4	5	$9.00	84
C2	Burger	4	4	5	5	6	7	8	$15.00	84
C3	Pasta	7	7	8	9	10	11	12	$18.00	76
C4	Salad	10	11	12	12	13	14	15	$9.00	76
C5	Burger	13	14	15	15	16	17	18	$15.00	76
C6	Pasta	16	17	18	-	-	-	-	-	-

Result: VICTORY
5 paid (need 4), revenue $66.00 (need $45.00), average satisfaction 79.2 (need 60). C6 arrives on turn 16 and cannot finish paying within 18 turns; the brief says this is intended.

The same run also tests invalid input on turn 3: the manager first tries to serve customer 2 (who has no order), then types `abc`. Both are rejected with a reason and the Waiter task is still available:

Console output, turn 3

```
Waiter task: 3
Customer number (e.g. 1 for C1): 2
  !! Customer 2 has no order. Please choose again.
Waiter: 1) Seat  2) Take order  3) Serve  0) Wait
Waiter task: abc
  !! 'abc' is not a number.
Waiter task: 3
Customer number (e.g. 1 for C1): 1
>> Waiter: C1 (Regular) is served Salad.
```

Defeat run (`stage-1/tests/defeat.txt`): the Waiter seats C1 and C2 but never takes an order. Seated customers lose 8 per turn: C1 leaves on turn 13 and C2 on turn 16, tables freed, orders never placed. 0 paid, $0.00 -> **DEFEAT**.

6. Stage 2 - Customer behaviour

> **Development assumption**
> Stage 2 has just arrived; Stage 3 is unknown. Extend the saved Stage 1 version.

New requirement

Customer	Satisfaction loss per waiting turn	Bill	Special behaviour (ours)
Regular	8	Full price	-
VIP	5	10% off	-
Critic	12	Full price	**Always orders the most expensive item on the menu** (to test the kitchen), instead of the normal food rule.

Which customers are which: in the base setting, **C2 (turn 4) is a VIP** and **C4 (turn 10) is a Critic**; the other four are Regular. That keeps six arrivals, as the brief requires, and every run contains at least one of each.

The key idea: polymorphism, not type checks

The brief says different behaviour must be *dispatched through the customer objects, not through a chain of type checks in the turn controller*. Because Stage 1 already called `loseSatisfaction()`, `chooseItem()` and `calculateBill()` on `Customer`, the two new types are just subclasses that override those methods:

VIPCustomer.java (new)

```
/** VIP: more patient (loses 5 per turn) and gets 10% off the bill. */
public class VIPCustomer extends Customer {

    public VIPCustomer(int id) {
        super(id);
    }

    @Override
    public String getType() { return "VIP"; }

    @Override
    public int getSatisfactionLoss() { return 5; }

    @Override
    public int calculateBill(int basePriceCents) {
        return Money.percent(basePriceCents, 90);
    }
}
```

CriticCustomer.java (new)

```
/**
 * Food critic: impatient (loses 12 per turn).
 * Special behaviour: always orders the most expensive item on the menu
 * (to test the kitchen), instead of following the normal choice rule.
 */
public class CriticCustomer extends Customer {

    public CriticCustomer(int id) {
        super(id);
    }

    @Override
    public String getType() { return "Critic"; }

    @Override
    public int getSatisfactionLoss() { return 12; }

    @Override
    public MenuItem chooseItem(Menu menu) {
        return menu.getMostExpensive();
    }
}
```

When the controller runs `c.loseSatisfaction()`, Java calls the right `getSatisfactionLoss()` for that object: 8, 5 or 12. When the Waiter calls `customer.calculateBill(1500)`, a VIP returns 1350. Nobody checks the type.

What changed from Stage 1 to Stage 2

File	Change	Why
`VIPCustomer.java`	**new**	Overrides `getSatisfactionLoss()` (5) and `calculateBill()` (10% off).
`CriticCustomer.java`	**new**	Overrides `getSatisfactionLoss()` (12) and `chooseItem()` (most expensive).
`ArrivalSchedule.java`	changed	Registration only: create a VIP for C2 and a Critic for C4.
`Menu.java`	changed	Added `getMostExpensive()` for the Critic's choice.
`Money.java`	changed	Added `percent()` (rounds to the nearest cent) for the VIP discount.
`tests/*.txt`	new	Reproducible runs that include a VIP and a Critic.
Unchanged	-	`GameController`, `ConsoleUI`, `Waiter`, `Chef`, `Cashier`, `Customer`, `Restaurant`, `Order`, `Table`, `Main`.

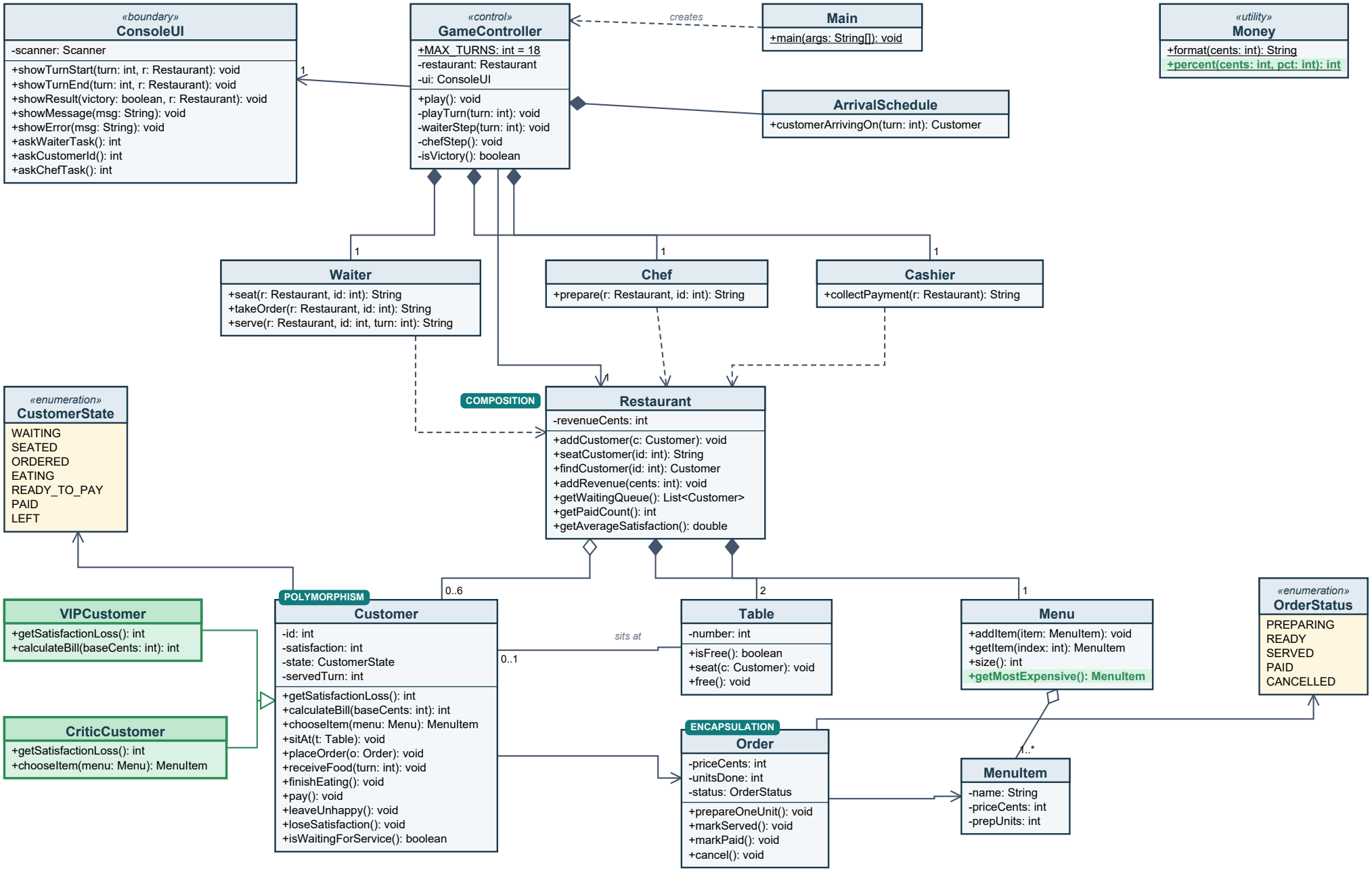
Size: 16 -> 18 classes, 656 -> 721 lines. More importantly, **no existing rule was edited**; every change is an addition. That is the Open/Closed Principle working: open for extension (new subclasses), closed for modification (the turn logic).

diff stage-1 -> stage-2: ArrivalSchedule.java (the only change to existing game logic)

```
@@ -1,2 +1,5 @@
-/** Decides who arrives on which turn: one customer on turns 1, 4, 7, 10, 13 and 16. */
+/**
+ * Decides who arrives on which turn: one customer on turns 1, 4, 7, 10, 13 and 16.
+ * C2 (turn 4) is a VIP and C4 (turn 10) is a Critic; the rest are Regular.
+ */
 public class ArrivalSchedule {
@@ -9,2 +12,8 @@
         int id = turn / 3 + 1; // turn 1 -> C1, turn 4 -> C2, ... turn 16 -> C6
+        if (id == 2) {
+            return new VIPCustomer(id);
+        }
+        if (id == 4) {
+            return new CriticCustomer(id);
+        }
         return new Customer(id);
```

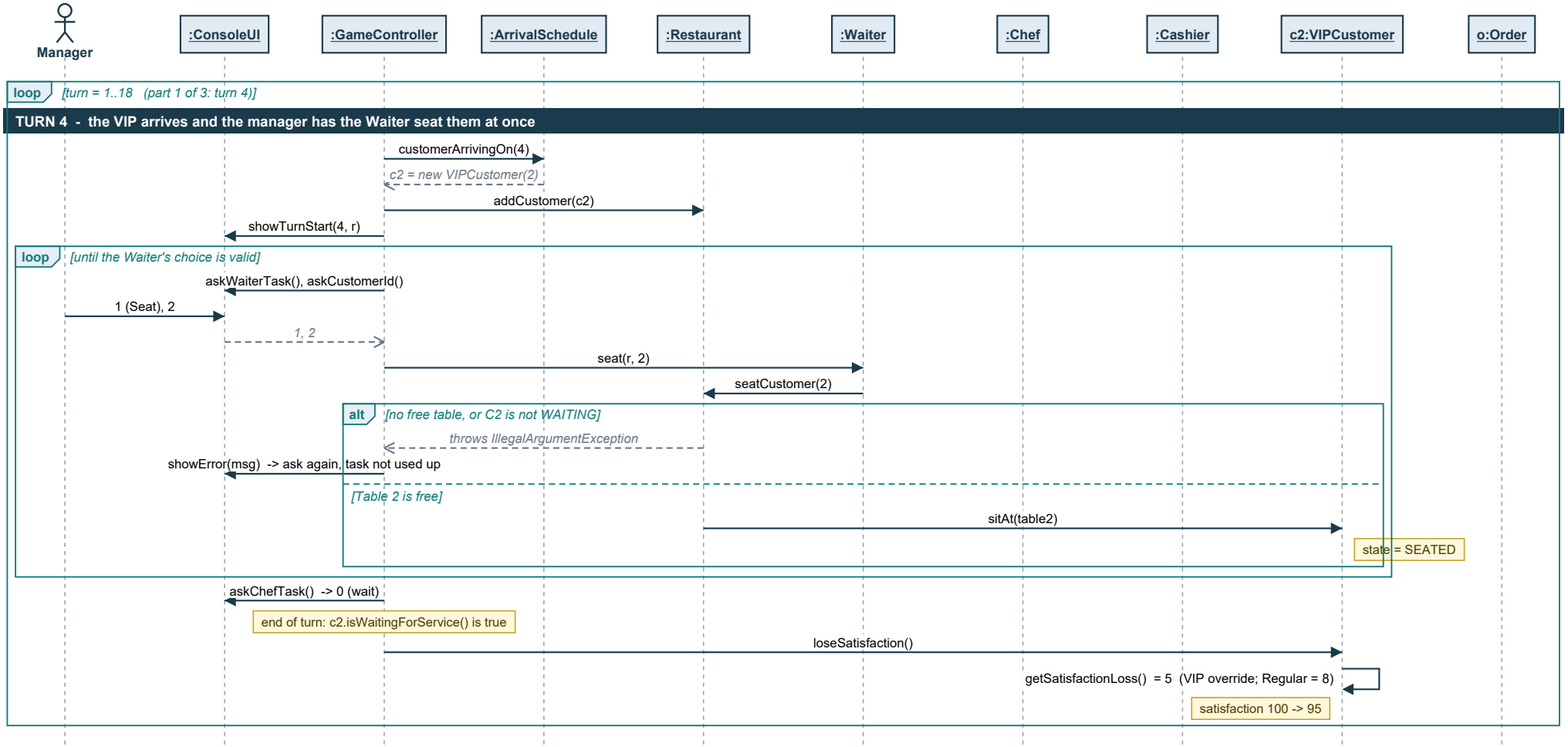
Stage 2 class diagram

*Matches `stage-2/src`. **Green** = new in Stage 2 (classes and members). Everything else is identical to Stage 1.*



Stage 2 sequence diagram - serving a VIP from arrival to payment

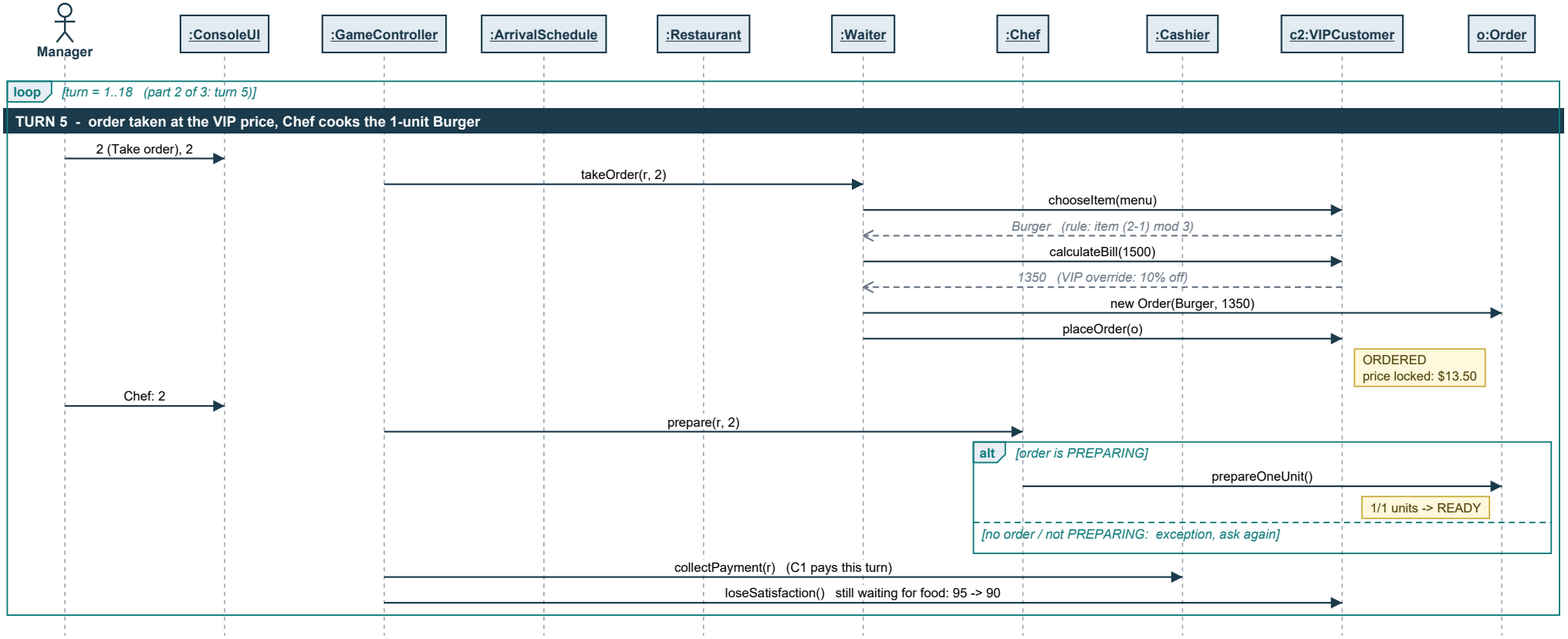
Use case: *Stage 2.* ***Starting conditions:*** *turn 4 of* `stage-2/tests/victory.txt`*; Table 2 is free and C1 is eating at Table 1.* ***Outcome:*** *VIP C2 pays $13.50 with satisfaction 90 on turn 8; after turn 18 the result is VICTORY. Part 1 of 3: turn 4.*



What this part shows. `GameController` asks `ArrivalSchedule` who arrives; on turn 4 that is a `VIPCustomer`. The **manager's choice** is to have the Waiter seat C2 immediately. The inner *loop* shows the validation rule: if there is no free table, `Restaurant.seatCustomer()` throws, the controller shows the reason, and the manager chooses again without losing the task. At the end of the turn the controller simply calls `loseSatisfaction()`; the VIP object answers with its own loss of 5 (polymorphism).

Stage 2 sequence diagram - part 2 of 3: turn 5

The order is taken at the VIP price and the Chef cooks it.

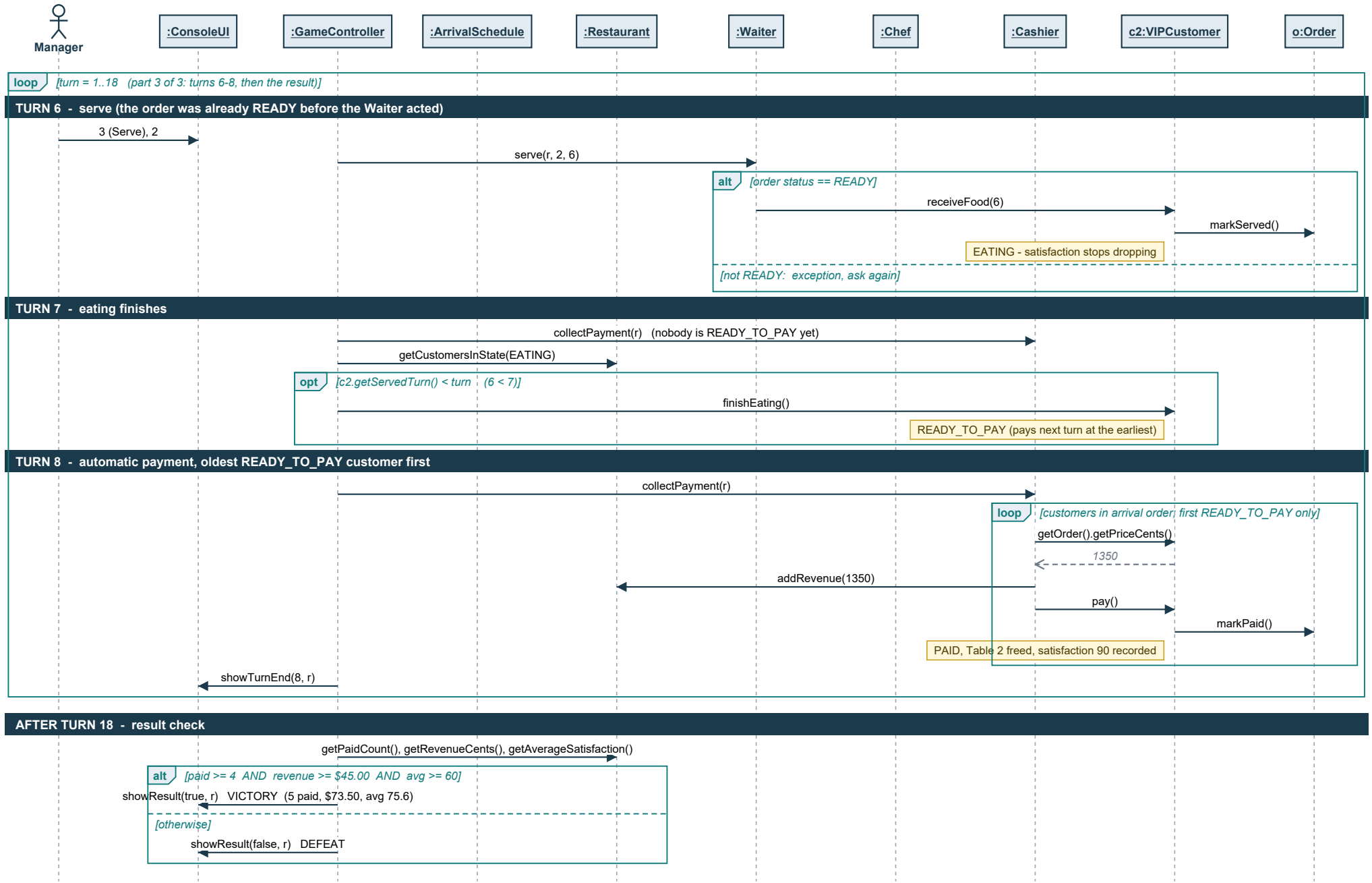


What this part shows. The Waiter asks the customer to choose (`chooseItem` gives Burger by the food rule) and to price the dish (`calculateBill(1500)`). The VIP override returns 1350, and the price is **locked** into the new `Order`. The Chef step runs after the Waiter, so the Chef can cook the order taken this turn; the *alt* guard shows the Chef only works on PREPARING orders. The Burger has one unit, so it becomes READY. C2 is still waiting for food, so they lose another 5 (95 -> 90).

How a manager choice changes the outcome. If the manager had made the Waiter wait on turn 4 instead of seating C2, every step would shift one turn later and C2 would lose 5 more satisfaction per turn of delay. Seating at once is what lets C2 pay on turn 8 with 90 - and in the full run it is what keeps the average above 60 after the impatient Critic (who loses 12 per turn) pays with only 52.

Stage 2 sequence diagram - part 3 of 3: turns 6 to 8 and the result check

Serve, eat, pay (oldest first), then after turn 18 the three targets are checked.


Manager
:ConsoleUI
:GameController
:ArrivalSchedule
:Restaurant
:Waiter
:Chef
:Cashier
c2:VIPCustomer
o:Order
loop [turn = 1..18 (part 3 of 3: turns 6-8, then the result)]
TURN 6 - serve (the order was already READY before the Waiter acted)
3 (Serve), 2
serve(r, 2, 6)
alt [order status == READY]
receiveFood(6)
markServed()
EATING - satisfaction stops dropping
[not READY: exception, ask again]
TURN 7 - eating finishes
collectPayment(r) (nobody is READY_TO_PAY yet)
getCustomersInState(EATING)
opt [c2.getServedTurn() < turn (6 < 7)]
finishEating()
READY_TO_PAY (pays next turn at the earliest)
TURN 8 - automatic payment, oldest READY_TO_PAY customer first
collectPayment(r)
loop [customers in arrival order, first READY_TO_PAY only]
getOrder().getPriceCents()
1350
addRevenue(1350)
pay()
markPaid()
PAID, Table 2 freed, satisfaction 90 recorded
showTurnEnd(8, r)
AFTER TURN 18 - result check
getPaidCount(), getRevenueCents(), getAverageSatisfaction()
alt [paid >= 4 AND revenue >= $45.00 AND avg >= 60]
showResult(true, r) VICTORY (5 paid, $73.50, avg 75.6)
[otherwise]
showResult(false, r) DEFEAT

How the sequence diagram meets the brief

Required in the sequence diagram	Where it is
Customer arrival and order	Part 1 (customerArrivingOn, addCustomer); part 2 (takeOrder, new Order)
Waiter and Chef assignments	Part 1 (seat), part 2 (takeOrder, prepare), part 3 (serve)
READY then SERVED	Part 2 (prepareOneUnit -> READY), part 3 (alt [status == READY] -> markServed)
Eating	Part 3, turn 7: opt [servedTurn < turn] -> finishEating()
Payment	Part 3, turn 8: loop over customers, first READY_TO_PAY pays, addRevenue, pay, markPaid
The result check	Part 3: alt [paid >= 4 AND revenue >= $45 AND avg >= 60] -> VICTORY / DEFEAT
One Stage 2 or 3 behaviour	VIP: getSatisfactionLoss() = 5 (part 1) and calculateBill(1500) = 1350 (part 2)
Messages among CLI, controller, staff and domain	Manager -> ConsoleUI -> GameController -> Waiter/Chef/Cashier -> Restaurant/Customer/Order
Guards and loops	loop (turns), loop (until valid), alt (seat / prepare / serve / result), opt (eating), loop (payment)

Stage 2 test runs

Customer	Type	Dish	Arrive	Seated	Ordered	Served	Paid	Bill	Satisf.
C1	Regular	Salad	1	1	2	3	5	$9.00	84
C2	**VIP**	Burger	4	4	5	6	8	**$13.50**	**90**
C3	Regular	Pasta	7	7	8	10	12	$18.00	76
C4	**Critic**	**Pasta**	10	11	12	14	16	$18.00	**52**
C5	Regular	Burger	13	13	15	16	18	$15.00	76
C6	Regular	Pasta	16	17	18	-	-	-	-

The VIP pays 10% less and loses satisfaction more slowly (90 instead of 84 for the same timing as C1/C2 in Stage 1). The Critic orders Pasta (the most expensive item, not Salad as the normal rule would give C4) and loses 12 per turn: 100 -> 88 -> 76 -> 64 -> 52 before being served.

> **Result: VICTORY**
> 5 paid, revenue $73.50, average satisfaction 75.6.

Defeat run (`stage-2/tests/defeat.txt`, same neglect as Stage 1): C1 leaves on turn 13. The VIP C2 loses only 5 per turn and is still seated (unhappy, not gone) at the end. The Critic C4, waiting in the queue from turn 10, loses 12 per turn and leaves on turn 18. 0 paid -> **DEFEAT**.

7. Stage 3 - New role and menu policy

> **Development assumption**
> Stage 3 has just arrived. Extend the saved Stage 2 version.

7.1 Host

The Host seats at most one queued customer per turn, **in addition to** the Waiter's task (the Waiter can still seat too). Seating rules were already in `Restaurant.seatCustomer()` since Stage 1 (the Waiter used it), so the Host reuses them and is one line long. The controller gets a `hostStep()` just like the Waiter and Chef steps, run before the Waiter.

Host.java (new)

```
/** The Host can seat ONE queued customer per turn, in addition to the Waiter's task. */
public class Host {

    public String seat(Restaurant restaurant, int customerId) {
        return restaurant.seatCustomer(customerId);
    }
}
```

7.2 ComboMeal

A combo is made of two existing menu items. Preparation units = the sum; price = 90% of the combined price, rounded to the nearest cent. Because `ComboMeal` **is a** `MenuItem` (inheritance) and **has** two MenuItems (composition), `Menu`, `Order` and `Chef` handle it with no changes at all.

ComboMeal.java (new)

```
/**
 * A combo is made of two existing menu items (composition).
 * Prep units = sum of both; price = 90% of both prices, rounded to the nearest cent.
 * Because it IS a MenuItem, the rest of the game (Menu, Order, Chef) uses it unchanged.
 */
public class ComboMeal extends MenuItem {
    private final MenuItem first;
    private final MenuItem second;

    public ComboMeal(MenuItem first, MenuItem second) {
        super(first.getName() + " + " + second.getName() + " Combo",
                Money.percent(first.getPriceCents() + second.getPriceCents(), 90),
                first.getPrepUnits() + second.getPrepUnits());
        this.first = first;
        this.second = second;
    }

    public MenuItem getFirst() { return first; }
    public MenuItem getSecond() { return second; }
}
```

Combo on the Stage 3 menu	Calculation	Price	Units
Salad + Pasta Combo	($9.00 + $18.00) x 0.9	$24.30	1 + 2 = 3
Burger + Salad Combo	($15.00 + $9.00) x 0.9	$21.60	1 + 1 = 2

How a customer chooses a combo (our documented rule): combos are simply added to the menu, so the normal food rule picks them: with five items, C4 lands on index 3 and C5 on index 4. In the base setting C4 is the Critic, who orders the most expensive item, which is now the Salad + Pasta Combo ($24.30). C5 orders the Burger + Salad Combo ($21.60).

7.3 Happy Hour

- The manager may switch it on **once per game**, at the start of a turn. It lasts for that turn and the next; the display shows the turns left.

- Orders **taken** while it is active get 20% off. The price is locked when the order is taken, so it never changes orders already placed or customers who have left.
- During each active turn, customers still waiting for a table or food recover 15 satisfaction at turn start (max 100).
- **VIP discount first, then Happy Hour.** Example: a VIP orders a Burger during Happy Hour: $15.00 x 0.9 x 0.8 = $10.80.

HappyHour.java (new)

```
/**
 * Pricing policy the manager can switch on ONCE per game, at the start of a turn.
 * It lasts for that turn and the next one. Orders taken while active get 20% off.
 */
public class HappyHour {
    public static final int RECOVERY = 15;

    private boolean used = false;
    private int turnsLeft = 0;

    public boolean canActivate() { return !used; }

    public void activate() {
        used = true;
        turnsLeft = 2; // this turn + next turn
    }

    public boolean isActive() { return turnsLeft > 0; }
    public int getTurnsLeft() { return turnsLeft; }

    /** Called at the end of every turn. */
    public void endTurn() {
        if (turnsLeft > 0) {
            turnsLeft--;
        }
    }

    /** 20% off while active. Applied AFTER the customer's own discount (e.g. VIP). */
    public int applyDiscount(int priceCents) {
        return isActive() ? Money.percent(priceCents, 80) : priceCents;
    }
}
```

The only rule that had to change: pricing in the Waiter

diff stage-2 -> stage-3: Waiter.java

```
@@ -14,5 +14,7 @@
         MenuItem item = customer.chooseItem(restaurant.getMenu());
-        int price = customer.calculateBill(item.getPriceCents()); // each customer type prices its own bill
+        int price = customer.calculateBill(item.getPriceCents());    // customer discount first (VIP)
+        price = restaurant.getHappyHour().applyDiscount(price);      // then Happy Hour
         customer.placeOrder(new Order(item, price));
-        return customer + " orders " + item.getName() + " for " + Money.format(price) + ".";
+        String note = restaurant.getHappyHour().isActive() ? " (Happy Hour price)" : "";
+        return customer + " orders " + item.getName() + " for " + Money.format(price) + note + ".";
     }
```

New steps in the turn

diff stage-2 -> stage-3: GameController.playTurn()

```
@@ -11,3 +11,19 @@

-    // Steps 2 + 3: manager chooses; Waiter acts before Chef
+    // Start of turn: Happy Hour (can be switched on once per game)
+    HappyHour happyHour = restaurant.getHappyHour();
+    if (happyHour.canActivate() && ui.askHappyHour()) {
+        happyHour.activate();
+        ui.showMessage(">> Happy Hour is ON for this turn and the next: 20% off new orders.");
+    }
+    if (happyHour.isActive()) {
+        for (Customer c : restaurant.getCustomers()) {
+            if (c.isWaitingForService()) {
+                c.recoverSatisfaction(HappyHour.RECOVERY);
+                ui.showMessage(">> Happy Hour: " + c + " recovers to satisfaction " + c.getSatisfaction() + ".");
+            }
+        }
+    }
+
+    // Steps 2 + 3: manager chooses; Host and Waiter act before Chef
+    hostStep();
     waiterStep(turn);
@@ -37,2 +53,3 @@
     }
+    happyHour.endTurn();
     ui.showTurnEnd(turn, restaurant);
```

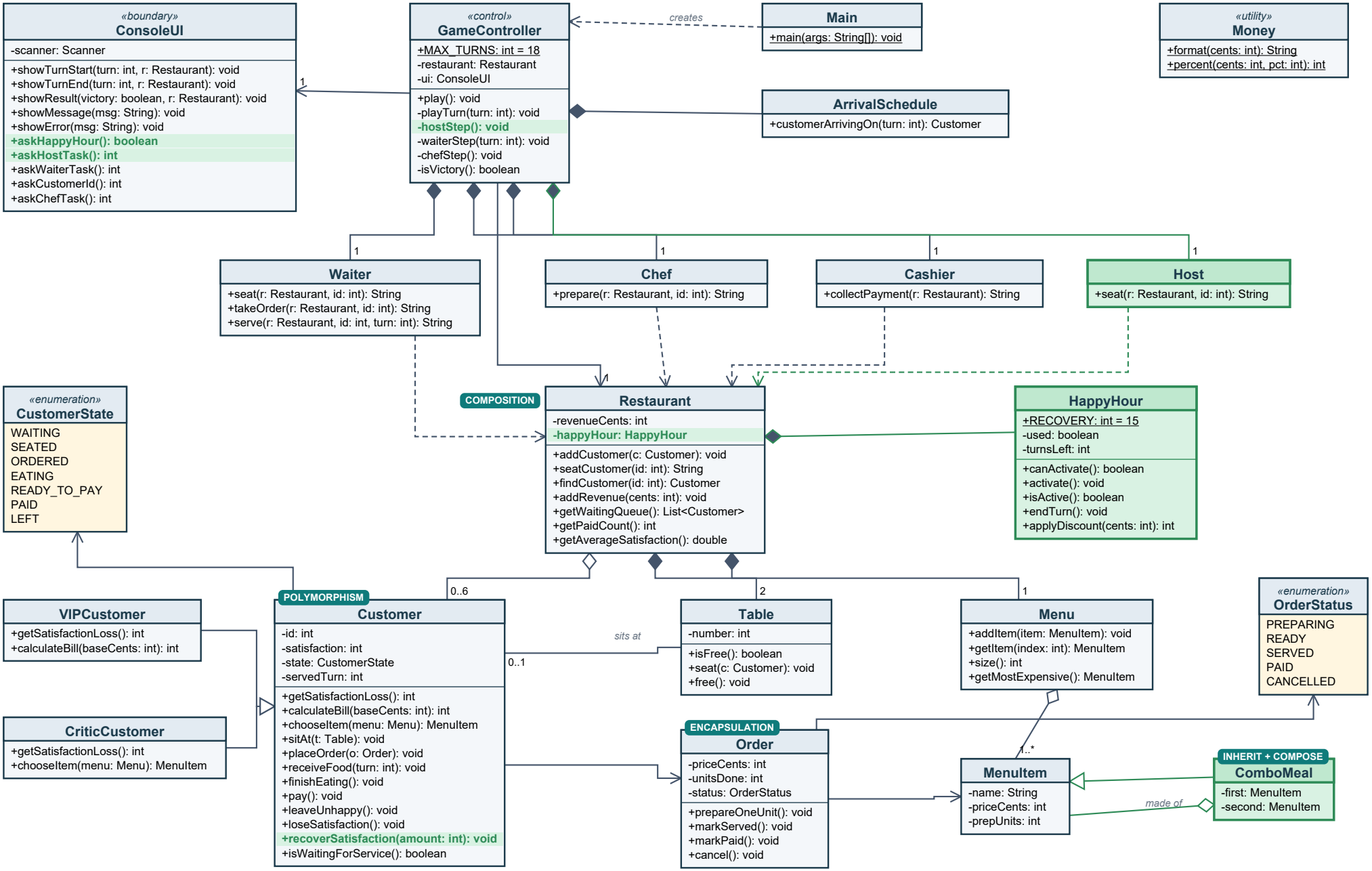
What changed from Stage 2 to Stage 3

File	Change	Why
`Host.java`	**new**	Seats one queued customer per turn; reuses `Restaurant.seatCustomer()`.
`ComboMeal.java`	**new**	Extends `MenuItem`, built from two items.
`HappyHour.java`	**new**	The once-per-game, two-turn pricing policy.
`GameController.java`	changed	Two new steps in the turn (Happy Hour at turn start, Host before Waiter) and `happyHour.endTurn()`. Unavoidable: these are new steps.
`ConsoleUI.java`	changed	New prompts (Happy Hour y/n, Host) and the Happy Hour status line.
`Waiter.java`	changed	`takeOrder()` applies Happy Hour after the customer's own discount.
`Restaurant.java`	changed	Owns the `HappyHour` object (it is part of the game state).
`Customer.java`	changed	Added `recoverSatisfaction()` (capped at 100).
`Main.java`	changed	Registration only: adds the two combos to the menu.
Unchanged	-	`Chef`, `Cashier`, `Order`, `Menu`, `MenuItem`, `Table`, `ArrivalSchedule`, `VIPCustomer`, `CriticCustomer`, `Money`.

Size: 18 -> 21 classes, 721 -> 852 lines. No stable rule was rewritten: payment, eating and satisfaction timing are the same code as in Stages 1 and 2, which the Stage 3 runs confirm.

Stage 3 class diagram (final design)

*Matches `stage-3/src`. **Green** = new in Stage 3. Tags mark the OO examples: ENCAPSULATION (Order), COMPOSITION (Restaurant owns Menu, Tables, HappyHour), POLYMORPHISM (Customer subclasses), INHERIT + COMPOSE (ComboMeal).*



Stage 3 test runs

Customer	Type	Dish	Seated by	Ordered	Served	Paid	Bill	Satisf.
C1	Regular	Salad	Host, t1	1	2	4	$9.00	92
C2	VIP	Burger	Host, t4	4	5	7	$13.50	95
C3	Regular	Pasta	Host, t7	7 (**Happy Hour**)	9	11	**$14.40**	92
C4	Critic	**Salad+Pasta Combo**	Host, t10	10	13	15	**$24.30**	64
C5	Regular	**Burger+Salad Combo**	Host, t13	14	16	18	**$21.60**	76
C6	Regular	Salad	Host, t16	17	18	-	-	-

- The Host seats every customer on arrival, which frees the Waiter to take the order **in the same turn**. That is why satisfaction is higher than in Stage 2.
- Happy Hour is switched on at turn 7: C3's Pasta is $18.00 x 0.8 = $14.40, and C3 recovers 15 at the start of turns 7 and 8 (capped at 100), ending at 92 instead of 84.
- Combos flow through Order and Chef like any dish: the 3-unit combo takes the Chef three turns (10, 11, 12).

> **Result: VICTORY**
> 5 paid, revenue $82.80, average satisfaction 83.8.

Defeat run (`stage-3/tests/defeat.txt`): Happy Hour declined every turn, the Host seats C1 and C2 and nobody takes an order. C1 leaves on turn 13, the Critic C4 (in the queue) on turn 18. 0 paid -> **DEFEAT**.

What one Stage 3 turn looks like on screen

stage-3 console, turn 7

```
================ TURN 7 / 18 ================
Revenue: $9.00   Avg satisfaction (paid): 92.0
Happy Hour: available (once per game)
Waiting queue: C3 (Regular) sat 100
Table 1: free
Table 2: C2 (VIP) READY_TO_PAY, satisfaction 95, order: Burger $13.50 [SERVED]
Staff tasks available:
  Host   -> seat: C3 (Regular) sat 100
  Waiter -> seat: C3 (Regular) sat 100 | take order: - | serve: -
  Chef   -> cook: -
>> C3 (Regular) arrives and joins the queue.
Start Happy Hour now? (y/n): y
>> Happy Hour is ON for this turn and the next: 20% off new orders.
>> Happy Hour: C3 (Regular) recovers to satisfaction 100.
Host: customer number to seat (0 = wait): 3
>> Host: C3 (Regular) is seated at Table 1.
Waiter task: 2
Customer number (e.g. 1 for C1): 3
>> Waiter: C3 (Regular) orders Pasta for $14.40 (Happy Hour price).
Chef: customer number whose order to cook (0 = wait): 3
>> Chef works on C3 (Regular)'s Pasta $14.40 [PREPARING 1/2].
>> Cashier: C2 (VIP) pays $13.50 (satisfaction 95). Table freed.
--- End of turn 7: served 2, unhappy departures 0, revenue $22.50, avg satisfaction 93.5, turns used 7/18 ---
```

8. The three stages side by side

Class	Stage 1	Stage 2	Stage 3
`Main`	**yes**	**yes**	**changed** (combos added)
`GameController`	**yes**	**yes**	**changed** (Host + Happy Hour steps)
`ConsoleUI`	**yes**	**yes**	**changed** (2 prompts + status)
`Restaurant`	**yes**	**yes**	**changed** (owns HappyHour)
`ArrivalSchedule`	**yes**	**changed** (VIP, Critic)	**yes**
`Customer`	**yes**	**yes**	**changed** (recoverSatisfaction)
`Waiter`	**yes**	**yes**	**changed** (Happy Hour price)
`Menu`	**yes**	**changed** (getMostExpensive)	**yes**
`Money`	**yes**	**changed** (percent)	**yes**
`Chef`, `Cashier`, `Table`, `Order`, `MenuItem`, enums	**yes**	**yes**	**yes**
`VIPCustomer`, `CriticCustomer`	-	**NEW**	**yes**
`Host`, `ComboMeal`, `HappyHour`	-	-	**NEW**
Classes / lines	16 / 656	18 / 721	21 / 852
Victory run	5 paid, $66.00, 79.2	5 paid, $73.50, 75.6	5 paid, $82.80, 83.8

Effect on the design (not just line counts)

Transition	Added	Modified (and why)	What still works
Stage 1 -> 2	Two Customer subclasses.	Only registration (ArrivalSchedule) and two small helpers (Menu, Money). The controller and staff did not change because they already talked to Customer through overridable methods.	Same turn logic: in the Stage 2 victory run, Regular customers C1 and C3 follow exactly the same timeline and end satisfaction as in Stage 1.
Stage 2 -> 3	A staff role, a menu item type, a pricing policy.	The controller gained two steps (they are new steps in the turn, so it must). The Waiter gained one pricing line. Everything about cooking, serving, eating and payment is untouched.	VIP pricing, Critic choice and satisfaction loss are unchanged; payment/eating timing still follows the same rules.

Comparing the stages yourself

```
diff -r stage-1/src stage-2/src
diff -r stage-2/src stage-3/src
```

On GitHub: commit `stage-1/` and tag it `stage-1`, add `stage-2/` and tag `stage-2`, add `stage-3/` and tag `stage-3`; then use the compare pages `.../compare/stage-1...stage-2` and `.../compare/stage-2...stage-3`.

9. OO concepts and design principles

Object-oriented concepts

Concept	Where it really happens in our code
Encapsulation	All fields are `private`. An Order's status only changes through `prepareOneUnit()`, `markServed()`, `markPaid()`, `cancel()`; its price is `final` (locked when the order is taken). Satisfaction can't leave 0..100 because it only changes through `loseSatisfaction()` / `recoverSatisfaction()`.
Inheritance	`VIPCustomer` and `CriticCustomer` extend `Customer`; `ComboMeal` extends `MenuItem`.
Polymorphism	The controller calls `c.loseSatisfaction()` and the Waiter calls `chooseItem()` / `calculateBill()` on any customer; the right version runs. There is no `instanceof` anywhere in the game logic.
Composition	`Restaurant` creates and owns its Tables (and in Stage 3 its HappyHour); `GameController` owns its staff; a `ComboMeal` is built from two MenuItems.

SOLID - only where it is genuinely in the code

Principle	Evidence (or an honest 'not really')
SRP	UI only does input/output; GameController only does the order of steps; each staff class does one job; HappyHour only does the promotion; ArrivalSchedule only decides arrivals.
OCP	Stage 2 added two customer types without changing GameController, Waiter, Chef or Cashier. Stage 3's ComboMeal needed no change to Order, Chef or Menu.
LSP	A VIPCustomer, CriticCustomer or ComboMeal can be used anywhere its parent is expected; every customer type follows the same order and service flow.
ISP / DIP	Not really applied, on purpose. The brief says not to create interfaces just to tick a box, and each role has one implementation. GameController creates concrete staff objects. This is a known trade-off (see below).

Trade-offs and an alternative we considered

- **Alternative: Command pattern for staff tasks.** Each action would be an object (`SeatTask`, `ServeTask` ...) with `isValid()` and `execute()`. More extensible (undo, new roles, AI player) but many small classes for a fixed set of four roles. We chose plain methods on each staff class.
- **Alternative: a `PricingPolicy` interface for Happy Hour.** Would allow more promotions later, but there is only one policy, so a single class is simpler and easier to read.
- **Derived queue** is recomputed each time it is needed; perfectly fine for six customers, would need an index for thousands.
- **Exceptions for invalid choices** keep each rule next to the code it protects, at the cost of using exceptions for expected user mistakes.

How the player's choices change the result

- **Seat vs order vs serve:** the Waiter has one task per turn. Serving a READY dish stops that customer losing satisfaction; taking an order starts the kitchen sooner. Picking the wrong one costs satisfaction.
- **Kitchen priority:** the Chef does one unit per turn. Cooking the impatient Critic first protects the average.
- **Happy Hour timing (Stage 3):** cheaper bills (less revenue) but +15 satisfaction for everyone still waiting. Switching it on when a 2-unit Pasta is about to be ordered keeps C3 at 92 instead of 84.
- **Doing nothing:** the defeat runs show customers draining to 0 and leaving without paying.

10. Every rule in the brief, and where the code enforces it

Rule	Enforced by
The Waiter completes at most one task per turn.	`waiterStep()` returns after the first successful task.
An invalid choice explains why and lets the manager choose again, without using up the task.	Staff methods throw `IllegalArgumentException(reason)`; the step loop shows it and asks again.
Non-numeric input is handled.	`ConsoleUI.readInt()` re-asks; end of input counts as 0 (wait).
A table stays occupied until its customer pays or leaves.	Only `Customer.pay()` and `leaveUnhappy()` call `table.free()`.
Seat only queued customers, only at a free table.	`Restaurant.seatCustomer()` (used by Waiter and Host).
Take an order only from a seated customer.	`Waiter.takeOrder()` checks state SEATED.
The Chef only cooks orders still being prepared.	`Chef.prepare()` checks status PREPARING.
Serve only orders that were already READY when the Waiter acts.	`Waiter.serve()` checks READY; Waiter runs before Chef.
A customer cannot eat before being served.	Only EATING customers can `finishEating()`; EATING is set by `receiveFood()`.
The Cashier cannot charge before READY_TO_PAY; one payment per turn; oldest first.	`Cashier.collectPayment()` returns after the first READY_TO_PAY in arrival order.
Revenue never includes unpaid or abandoned orders.	Revenue is only added in `collectPayment()`.
Waiting/seated/unserved customers lose satisfaction (min 0); at 0 they leave and the order is cancelled.	`isWaitingForService()`, `loseSatisfaction()` (Math.max 0), `leaveUnhappy()`.
Average uses the satisfaction recorded at payment; N/A if nobody paid.	Satisfaction stops changing once served; `getAverageSatisfaction()` returns -1 -> shown as N/A.
Always 18 turns, no early ending.	`play()` loops `turn = 1..MAX_TURNS`.
End-of-turn results: served, departures, revenue, average, turns used.	`ConsoleUI.showTurnEnd()`.
Money stored as integer cents, shown with two decimals.	`int priceCents`, `Money.format()`, `Money.percent()`.
Rules out of the UI.	ConsoleUI only displays and reads; every check is in staff / domain / controller classes.
Behaviour dispatched through customer objects, not type checks (Stage 2).	Overridden methods on Customer subclasses; no `instanceof`.
VIP discount first, then Happy Hour; price locked at order time (Stage 3).	`Waiter.takeOrder()` then `new Order(item, price)` with a `final` price.
Happy Hour once per game, this turn + next, remaining duration shown.	`HappyHour.canActivate() / activate() / endTurn()`; `showTurnStart()`.
Works offline.	Plain Java, no libraries, no network.

11. Running the game and the submission checklist

Build and run any stage

```
cd stage-1           (or stage-2, stage-3)
javac -d out src/*.java
java -cp out Main                       play interactively
java -cp out Main < tests/victory.txt   reproducible victory
java -cp out Main < tests/defeat.txt    reproducible defeat
```

Java 17 or newer, no internet, no libraries. Each stage folder is a complete, independent game.

How to play

Prompt	What to type
(Stage 3) Start Happy Hour now? (y/n)	`y` or `n` - only asked until you have used it once.
(Stage 3) Host: customer number to seat	e.g. `3` for C3, or `0` to wait.
Waiter task	`1` seat, `2` take order, `3` serve, `0` wait; then the customer number.
Chef	the customer number whose order to cook, or `0` to wait.

Project layout

```
RestaurantRush/
  stage-1/src, stage-1/tests     Stage 1 game + victory/defeat scripts
  stage-2/src, stage-2/tests     Stage 2 game
  stage-3/src, stage-3/tests     Stage 3 game
  docs/
    class-diagram-stage1.mmd     class diagrams (Mermaid source; GitHub renders them)
    class-diagram-stage2.mmd
    class-diagram-stage3.mmd
    sequence-diagram-stage2.mmd  the Stage 2 use case
    diagrams.md                  all diagrams with explanations
    view-diagrams.html           renders the Mermaid diagrams, PNG/SVG download
    REPORT.md                    report draft
    Restaurant_Rush_Guide.pdf    this document
```

Submission checklist (from the briefing)

Item	Status	Where / what is left
Runnable code for all 3 stages	**done**	`stage-1/`, `stage-2/`, `stage-3/`; all six scripted runs give the expected result.
3 class diagrams (one per stage, matching the code)	**done**	Sections 5, 6, 7 of this guide; `docs/class-diagram-stage*.mmd`.
1 Stage 2 or 3 sequence diagram	**done**	Section 6 (Stage 2, VIP use case); `docs/sequence-diagram-stage2.mmd`.
Stage comparisons (added / modified / still works)	**done**	Sections 6, 7, 8.
GitHub tags stage-1, stage-2, stage-3 on runnable commits	**to do**	Create the repo, commit stage by stage, tag each; add the compare links to the README.
Group report with individual reflections	**to do**	Use this guide and `docs/REPORT.md`; each member writes their own reflection (learned, applied, challenge, solution).
15-minute presentation video	**to do**	2 min overview, 4 min gameplay, 4 min class diagrams + changes, 3 min sequence diagram, 2 min contributions; timestamps in the README.
Full AI histories on each member's branch	**to do**	`ai-history/stage-N/chat-01.md` on each personal branch; README links to all of them.
Declarations; check all links are accessible	**to do**	One submission per group.